BSc Thesis: Code Extraction from Production HPC Codebases and Cross-Representation Performance Analysis For OptArena

Student name: Hulsbergen Lars
Advisors: Lorenzo Paleari lorenzo.paleari@inf.ethz.ch, Yakup Koray Budanaz, ybudanaz@ethz.ch
Professor: Prof. Dr. T. Hoefler
Start time: 01.07.2026
End time: 30.11.2026

The final report is to be submitted electronically. All developed IP, copies of written documents, and contributed software remain property of the Scalable Parallel Computing Laboratory. If the thesis contains performance experiments, those should follow rules in the paper "Scientific Benchmarking of Parallel Computing Systems". Note that the latency between submitting your thesis (which includes defending it) and grading is up to three weeks.

1. Introduction

AgentBench is a benchmark for evaluating agentic code optimizers across three tracks: (i) Foundation (classic compiler-optimization puzzles), (ii) HPC (>=500-LoC sub-computations from production codebases such as ICON and Quantum ESPRESSO), and (iii) ML (KernelBench-derived, pure Python/NumPy). It generates inputs and checks correctness against a NumPy oracle, and runs agents in isolation.

Existing efforts measure adjacent targets, either benchmark only ML kernels (MLBench), or target software engineer and repository-level engineer (SWE-bench), but none measure the numerical-performance optimization on production-derived HPC kernels.

This thesis will help the kernel extraction effort of AgentBench, and evaluate the performance variation of the same computation across different baseline representations (C, C++, Fortran, and DaCe-emitted C++).

2. Project description

Part I of the thesis will extract large, representative computations from production HPC codebases (e.g., from GB award-winning code repositories such as Quantum Espresso, or mini-apps such as CloudSC) into the AgentBench HPC track.

Part II will compare the same kernels across program representations (original Fortran/C/C++; a Python port lowered back to C, C++, and Fortran via the NumpyToX translators (NumpyToX translators are already integrated into AgentBench); and DaCe SDFG-emitted C++ generated from DaCe Fortran or Python frontends) and analyze why compilers emit different code for them.

Tables **1: Candidate codebases**, **2a: GB winners**, **2b: GB for Climate Modelling Winners**, **3: Example Benchmarks Classified per HPC-Dwarf** are provided as auxiliary material at the end of the document.

3. Project Plan (Milestones)

WP1 — Small-benchmark extraction (Month 1)

- Set up AgentBench and the toolchains.

- Extract small kernels from benchmarking suites, such as Rodinia or PARSEC (See Tables 1 and 3 for guidance), classify them according to the 13 dwarfs of HPC (https://dl.acm.org/doi/10.1145/2188286.2188341), and integrate to AgentBench.

WP2 — HPC codebase extraction and integration (Months 2–4)

- Survey HPC codebases, categorize them according to the 13 dwarfs of HPC, and add select hot-loops nests for extraction. (See Table 2)
- After the selection, isolate the hot loop nests from the repositories, and reimplement them as standalone NumPy kernels.
- Emit C and Fortran variants using the Numpy translator integrated into AgentBench.

WP3 — Benchmark Evaluation (Months 4–5)

- Build the representation matrix per core (original code; NumpyToX-lowered C, C++, Fortran; DaCe SDFG C++ generated from both Fortran and Python frontends) and verify numerical equivalence.
- Benchmark every representation on CPU and GPU.
- Attribute the dominant performance gaps to specific optimization decisions by inspecting the emitted IR/assembly (aliasing, array semantics, applied/failed transformation, library calls, loop structure, auto vectorization/parallelization).

WP4 — Finalization (Month 6)

- Finalize the benchmarks; complete packaging and documentation; write the final report; prepare contributions as PRs for upstreaming under BSD-3.

4. Project administration

The student is advised, but not required, to write a weekly report at the end of each week and to send it to his advisors. The idea of the weekly report is to briefly summarize the work, progress and any findings made during the week, to plan the actions for the next week, and to bring up open questions and points. The weekly report is also an important means for the student to get a goal-oriented attitude to work. The student may write a monthly report to the whole team, including professors as a self-contained email with embedded figures with approximately 1, maximum 2 pages of text (if many figures). The monthly report email must be self-contained (no links/attached pdf documents) with inline figures.

The final report has to be presented at the end of the project and a digital copy needs to be handed in and remain property of the SPCL. Note that this task description is part of your report and has to be attached to your final report. Note that we will need up to two weeks after the presentation to submit the final grade.

5. Deliverables

1. A set of new AgentBench HPC-track benchmarks and integration (NumPy reference and C/C++/Fortran baselines), contributed upstream as pull requests under BSD-3.
2. A compiler code-generation attribution report mapping the dominant performance gaps to concrete compiler decisions.
3. If found, DaCe/AgentBench bug reports and minimal reproducers opened as issues.
4. The final thesis document; reproduction scripts and archived raw measurement data.

Software deliverables should be licensed under a permissive open-source license, such as BSD-3.

5. Success criteria

The milestones for the work packages 1 to 4 are mostly accomplished.

Lars Hulsbergen

6. Auxiliary Material

Table 1. Example Code Repositories to Extract From

Source	Kernel(s)	Pattern	Kernel File(s)
CloudSC [Yakup is working on it] (ECMWF) repo	cloudsc	Structured Grid	src/cloudsc_fortran/cloudsc.F90
COSMO [Done] COSMO-ORG/cosmo repo	hdiff, vadvc	Structured Grid	
ICON [Yakup is working on it] (DWD/MPI-M) repo	velocity_tend.z_v_grad_w	Unstructured Grid	
miniWeather mrnorman/miniWeather repo	compute_tend_x, compute_tend_z	Structured Grid	c/miniWeather_serial.cpp
GROMACS gromacs/gromacs repo	nbnxm nonbonded	N-Body	src/gromacs/nbnxm/
ExaMiniMD ECP-copa/ExaMiniMD repo	force_lj	N-Body	src/force_lj_neigh.h
DBCSR (CP2K) repo	batched smallGEMM	Dense LA	src/mm/dbcsr_mm.F
Quantum ESPRESSO QEF/q-e repo	vloc_psi, addusxx_g	Spectral Methods	PW/src/vloc_psi_k.f90 *(Supervisor will provide serialized data)*
Grid (LQCD) repo	Dslash (Wilson/DWF)	Structured Grid	benchmarks/Benchmark_dwf.cc
LULESH [Yakup is working on it] LLNL/LULESH repo	CalcForce, CalcHourglass	Unstructured Grid	lulesh.cc
XSBench XSBench repo	calculate_macro_xs	Monte Carlo	openmp-threading/Simulation.c
CloverLeaf UK-MAC/CloverLeaf repo	advec_cell, PdV, flux_calc	Structured Grid	src/flux_calc.f90
sw4lite geodynamics/sw4lite repo	rhs4th3fort	Structured Grid	src/rhs4th3fort.f
PIConGPU ComputationalRadiation Physics repo	Esirkepov	N-Body	particles/pusher/Boris.hpp currentDeposition/Esirkepov.hpp
miniFE Mantevo/miniFE repo	CSR SpMV	Sparse LA	openmp-opt/src/SparseMatrix_functions.hpp
BookLeaf UK-MAC/BookLeaf_Cpp repo	getacc, getforce	Unstructured Grid	src/packages/hydro/kernel/get_force.cpp
NekRS / CEED Nek5000/nekRS repo		Dense LA	CEED/libCEED benchmarks/

Source	Kernel(s)	Pattern	Kernel File(s)
PARSEC (Princeton) repo	blackscholes	Map	pkgs/apps/blackscholes/src/blackscholes.c
PARSEC (Princeton) repo	streamcluster	Clustering	pkgs/kernels/streamcluster/src/streamcluster.cpp
Rodinia (UVA) repo	lavaMD	N-Body	cuda/lavaMD/kernel/kernel_gpu_cuda.cu openmp/lavaMD/main.c
Rodinia (UVA) repo	srad	Structured Grid	cuda/srad/srad_v2/srad.cu openmp/srad/srad_v2/srad.cpp

Table 2a. ACM Gordon Bell Prize winners (2015–2025).

Year	Project	Repository
2015	Implicit solver, Earth's mantle flow	Rhea (not public); p4est + PETSc
2016	Fully-implicit atmospheric dynamics	Sunway code (not public)
2017	Nonlinear earthquake simulation	Sunway code (not public)
2018 (shared)	Opioid-epidemic genomics (CoMet)	github.com/wdj/comet
2018 (shared)	Exascale DL for climate analytics	github.com/mlcommons/hpc (DeepCAM)
2019	Data-centric quantum transport	github.com/spcl/dace; github.com/saschabrueck/dft-transport
2020	Ab-initio MD, 100M atoms with ML	github.com/deepmodeling/deepmd-kit
2021	Random quantum circuit simulation	Sunway code (not public)
2022	Mesh-refined PIC accelerators	github.com/ECP-WarpX/WarpX
2023	Materials modeling at quantum accuracy	github.com/dftfeDevelopers/dftfe
2024	Million-electron MP2 ab-initio MD	EXESS (not public)
2025	Tsunami early-warning digital twin	No public repo
2025 (Hon. M.)	GW quantum transport, 42,240 atoms	github.com/Nano-TCAD/QuaTrEx

Table 2b. ACM Gordon Bell Prize for Climate Modelling winners (2023–2025).

Year	Project	Repository
2023 (first)	SCREAM cloud-resolving atmosphere	github.com/E3SM-Project/E3SM
2024	Exascale climate emulator (storage)	github.com/ecrc/exageostat (PaRSEC)
2025	Full Earth system at 1.25 km (ICON)	icon-model

Table 3. Example Benchmarks per Berkeley dwarf.

Dwarf	Repository	Does	Candidate core (example)
1. Dense Linear Algebra	netlib.org/benchmark/hpl	LINPACK	HPL_pdgesv()
2. Sparse Linear Algebra	github.com/hpcg-benchmark/hpcg	SpMV	ComputeSPMV()
3. Spectral Methods	nas.nasa.gov/software/npb (FT)	FFT	cffts1()
4. N-Body Methods	github.com/ECP-copa/ExaMiniMD	MD	ForceLJ::compute()
5. Structured Grids	github.com/mrnorman/miniWeather	stencil	compute_tendencies_x()
6. Unstructured Grids	github.com/LLNL/LULESH	FEM	CalcForceForNodes()
7. Monte Carlo (a.k.a. MapReduce)	github.com/ANL-CESAR/XSBench	transport	calculate_macro_xs()
8. Combinational Logic	github.com/vtsynergy/OpenDwarfs (crc)	CRC	crc()
9. Graph Traversal	github.com/sbeamer/gapbs	BFS	DOBFS()

Dwarf	Repository	Does	Candidate core (example)
10. Dynamic Programming	github.com/vtsynergy/OpenDwarfs (nw)	align	needleman_wunsch()
11. Backtrack & Branch-and-Bound	github.com/vtsynergy/OpenDwarfs (nqueens)	search	nqueen_solve()
12. Graphical Models	github.com/vtsynergy/OpenDwarfs (hmm)	HMM	forward()
13. Finite State Machines	github.com/jackwadden/ANMLZoo	automata	automata_step()
2. Sparse LA	github.com/EgorOrachyov/SpBench	sparse	spgemm()
2. Sparse LA	github.com/HPC-Dwarfs/SparseBench	sparse	cg()
2. Sparse LA	CRAN: RHPCBenchmark	sparse	spmv()
?. Irregular / memory	github.com/hpcgarage/spatter	gather/scatter	gather()
?. Irregular / memory	icl.utk.edu/hpcc	memory	RandomAccess()
9. Graph	github.com/ParAlg/gbbs	graph	BFS()
9. Graph	github.com/jshun/ligra	graph	edgeMap()
9. Graph	github.com/IntelligentSoftwareSystems/Galois	graph	PageRank()
9. Graph	github.com/ldbc/ldbc_graphalytics	graph	PageRank()
9. Graph	github.com/chenxuhao/GraphAIBench	graph-ML	triangle_count()
9. Graph	github.com/spcl/gms	mining	k_clique()
?. Mesh / CFD / particle	github.com/geodynamics/sw4lite	stencil	rhs4sg()
?. Mesh / CFD / particle	github.com/Mantevo/miniAMR	AMR	stencil_calc()
?. Mesh / CFD / particle	github.com/Mantevo/miniAero	CFD	compute_face_flux()
?. Mesh / CFD / particle	github.com/opensbli/opensbli	stencil	residual()
?. Mesh / CFD / particle	github.com/sailfish-team/sailfish	LBM	collide()
?. Mesh / CFD / particle	github.com/ECP-copa/CoMD	MD	ljForce()